

Library components and APIs

CSC103 Programming Fundamentals / Lecture 27

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Sort $\{9,1,4\}$ without changing the original. Search the sorted result for 4 and explain the index.

Copy the original array before sorting.

Learning objectives

- Read an API signature and contract
- Use library methods without guessing side effects
- Choose and verify a suitable component

CLO-3

API contracts

Imports and qualification

Array utilities

Searching with a precondition

API contracts

- An API exposes operations that another program may call.
- Read parameter types, return type and documented failures.
- Check preconditions and side effects.

Example

```
Arrays.sort(int[] a)
```

```
Result type: void
```

```
Effect: sorts the supplied array
```

```
Caller-visible mutation: yes
```

Imports and qualification

- An import lets a source file use a shorter type name.
- It does not copy library code into the source.
- java.lang types such as Math are available without an explicit import.

Example

```
import java.util.Arrays;  
  
Arrays.sort(values);
```

Array utilities

- `Arrays.sort` rearranges elements into ascending order.
- `Arrays.copyOf` can create an independent primitive array.
- `Arrays.toString` provides readable element output.

Example

```
int[] copy = java.util.Arrays.copyOf(a, a.length);  
java.util.Arrays.sort(copy);
```

Searching with a precondition

- `Arrays.binarySearch` requires appropriately sorted input.
- A nonnegative result is a matching index.
- A negative result encodes an insertion point and means absent.

Example

```
Sorted input: {2,5,8}  
Search 5: index 1  
Search 7: negative result
```

Preserving order is a design requirement

- Sorting in place changes the supplied array.
- If the original sequence has meaning, copy the primitive elements before sorting.
- The sorted copy can then satisfy a search precondition without changing the original.

Example

Original sequence: {9,1,4}

Sorted copy: {1,4,9}

Original sequence remains {9,1,4}.

A search result must be interpreted by its contract

- `binarySearch` returns a matching index when the key exists.
- When absent, it returns $-(\text{insertion point})-1$.
- Never use a negative result directly as an array index.

Example

In `{1,4,9}`, key 6 has insertion point 2.
Returned value: $-2 - 1 = -3$.

Read an API contract before composing calls



What to notice

A library name is not a complete specification; mutation and failure behaviour matter too.

Key distinctions

Library call	Modifies input?	What it returns
<code>Arrays.sort(a)</code>	Yes	void
<code>Arrays.copyOf(a,n)</code>	No	A new array
<code>Arrays.toString(a)</code>	No	Readable String
<code>Arrays.binarySearch(a,key)</code>	No	Match index or encoded absence

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int[] data = {8, 2, 5};  
int[] copy = java.util.Arrays.copyOf(data, data.length);  
java.util.Arrays.sort(copy);  
System.out.println(java.util.Arrays.toString(copy));  
System.out.println(java.util.Arrays.binarySearch(copy, 5));  
System.out.println(java.util.Arrays.toString(data));
```

First example: result

[2, 5, 8]

1

[8, 2, 5]

Sorting the copy leaves the original order unchanged.

Array utilities

Searching with a precondition

Guided practice

Sort $\{9,1,4\}$ without changing the original. Search the sorted result for 4 and explain the index.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Copy the original array before sorting.
- Sort the copy in ascending order.
- Search the sorted array and interpret the resulting index.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution27 {  
    public static void main(String[] args) throws Exception {  
        int[] original = {9, 1, 4};  
        int[] sorted = java.util.Arrays.copyOf(original, original.length);  
        java.util.Arrays.sort(sorted);  
        int index = java.util.Arrays.binarySearch(sorted, 4);  
        System.out.println(java.util.Arrays.toString(sorted));  
        System.out.println(index);  
        System.out.println(java.util.Arrays.toString(original));  
    }  
}
```


Execution trace

Stage	Original	Copy or result
Before copy	{9,1,4}	None
After copy	{9,1,4}	{9,1,4}
After sort	{9,1,4}	{1,4,9}
Search for 4	Unchanged	Index 1

Follow the changing state in execution order.

Solution result

[1, 4, 9]

1

[9, 1, 4]

Why this result follows

Search the sorted array and interpret the resulting index.

A common incorrect approach

```
int[] a = {8,2,5};  
int index = java.util.Arrays.binarySearch(a, 2);
```

Example

The input violates the sorting precondition. Sort first, **while** considering whether changing the original order is acceptable.

Boundary and failure checks

Check the stated input assumptions.

Read documentation for `Arrays.equals` and `Arrays.copyOf`.

Explain their signatures and demonstrate each with a short program.

Copy, sort the copy to `{1,4,9}`, then `binarySearch` for 4 returns 1. The original stays `{9,1,4}`.

Check your understanding

Does importing Arrays sort an array? Is a negative `binarySearch` result a valid index?

Write a prediction and explain the rule that supports it.

Answer and explanation

No, an import only affects name resolution. No, a negative result means the key was not found.

The input violates the sorting precondition. Sort first, while considering whether changing the original order is acceptable.

Compare cases and predict outcomes

Arrays operation	Mutates input?	Important condition
<code>sort(a)</code>	Yes	Ascending order afterward
<code>binarySearch(a,key)</code>	No	Array must already be sorted
<code>copyOf(a,n)</code>	No	Returns a separate array

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Sort $\{9,1,5\}$, then search for 5 using `Arrays.binarySearch`. Explain why searching the unsorted array is not a valid alternative.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
int[] values = {9, 1, 5};  
java.util.Arrays.sort(values);  
int index = java.util.Arrays.binarySearch(values, 5);  
System.out.println(index);
```

- Sorting produces {1,5,9}.
- Searching for 5 returns index 1.
- `binarySearch` requires sorted input; an unsorted result cannot be relied upon.

Sorting library arrays

- `Arrays.sort` changes the supplied array in place.
- Character sorting follows numeric UTF-16 code-unit values, not human alphabetical order.
- Search positions refer to the sorted arrangement, not the original arrangement.

Source: supplied ISB campus slides. See speaker notes and [ISB_Source_Map.md](#).

Worked problem: Sorting library arrays

Task

Sort the numeric and character arrays supplied in the source example, then display them.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture27 {  
    public static void main(String[] args) throws Exception {  
        double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};  
        char[] chars = {'a', 'A', '4', 'F', 'D', 'P'};  
        java.util.Arrays.sort(numbers);  
        java.util.Arrays.sort(chars);  
        System.out.println(java.util.Arrays.toString(numbers));  
        System.out.println(java.util.Arrays.toString(chars));  
    }  
}
```

Worked trace and expected result

Element kind	Order used	Example
double	Ascending numeric value	1.9 before 6.0
char	UTF-16 code-unit value	4 before A
Search after sort	Sorted index	3.4 is at index 2

Expected console output

[1.9, 2.9, 3.4, 3.5, 4.4, 6.0]

[4, A, D, F, P, a]

Extend the ISB example

Practice

How do you preserve the original order before sorting?

Explain your reasoning

Copy the array first, for example with `Arrays.copyOf`, then sort the copy.

Lecture summary

- an API signature and contract
 - library methods without guessing side effects
 - and verify a suitable component
- Copy the original array before sorting.
 - Sort the copy in ascending order.
 - Search the sorted array and interpret the resulting index.

After-class practice

Read documentation for `Arrays.equals` and `Arrays.copyOf`.
Explain their signatures and demonstrate each with a short program.

Syllabus reading
Reference material
Next lecture: Files and streams